

```
let req = test::TestRequest::get().uri("/items").to_
request();
let resp = test::call_service(&app, req).await;
assert!(resp.status().is_success());
}
```

Unit tests test the individual handlers; integration tests the real API flows. This guarantees correctness without spinning up external infra.

Load testing (wrk/drill): Measure performance under stress after correctness. Tools like wrk or drill help to find bottlenecks and saturation points.

```
wrk -t4 -c100 -d30s http://127.0.0.1:8080/items
#[utoipa::path(
    get,
    path = "/items",
    responses(
        (status = 200, description = "List all items")
    )
)]
async fn list_items() {}
```

Deployment and distribution

The single binary advantage: One of the biggest strengths of Rust is that it produces a statically linked single binary. Your whole Actix Web API (and its dependencies) compiles into a single executable; there are no external runtimes or libraries.

```
cargo build --release
```

The result is a portable binary that you can copy and run anywhere (Linux servers, cloud VMs, etc). This reduces deployment complexity, eliminates ‘it works on my machine’ issues, and improves startup time because everything is precompiled.

Containerisation (Distroless/Alpine): Rust binaries are standalone, but containers help standardise deployments. We want the picture to be safe and small.

Here’s a minimal Alpine-based Dockerfile:

```
FROM rust:1.75 as builder
WORKDIR /app
COPY . .
RUN cargo build --release

FROM alpine:latest
WORKDIR /app
COPY --from=builder /app/target/release/ultra_api .
CMD ["./ultra_api"]
```

Monitoring (Prometheus metrics): Performance isn’t just

about deployment—you need to see what’s going on. Add Prometheus to monitor requests, latencies and health of the system in real time.

Using actix-web-prom:

```
use actix_web_prom::PrometheusMetrics;
let prometheus = PrometheusMetrics::new("api", Some("/metrics"), None);

App::new()
    .wrap(prometheus)
```

This exposes a `/metrics` endpoint for Prometheus to scrape. Then you can visualise data in tools such as Grafana.

Comparison and alternative perspectives (the ‘reality check’)

Actix Web vs Axum: Both are great frameworks but they are different design philosophies. Actix Web is mature, battle tested and highly optimised for raw performance. It ships with a lot of features out of the box like extractors, middleware and a well-defined request lifecycle.

Axum, on the other hand, is built on the tower ecosystem, focusing on composability and modular design. Its APIs seem more modern and fit well into the async ecosystem in Rust.

```
// Axum-style handler
async fn handler() -> &'static str {
    "Hello from Axum"
}
```

If you want maximum performance and stability, Actix Web is a great choice. If you want flexibility and ecosystem alignment, Axum might feel cleaner

When NOT to use Rust

To conclude let us see why we should not use Rust. Rust is powerful but not always the quickest way to deliver. For low-traffic internal tools, prototypes or scripts, languages like Go or Python can be more productive. Go provides easier concurrency and faster compile times. Python allows rapid prototyping with little boilerplate. **END** 🐧

By: Sufyan bin Uzayr

The author, apart from being a writer, is an educator and entrepreneur. He holds multiple degrees in management, information technology and humanities. He has written several books and is currently the director of a multinational IT company. As an educator, he has taught at universities and institutions worldwide.